

LẬP KẾ HOẠCH (PLANNING)

CHƯƠNG 11

Nội dung

- ◇ Tìm kiếm so với lập kế hoạch
- ◇ Các toán tử STRIPS
- ◇ Lập kế hoạch theo thứ tự cục bộ (Partial-order planning)

Tìm kiếm so với lập kế hoạch

Xem xét nhiệm vụ *lấy sữa, chuối và máy khoan không dây*

Các thuật toán tìm kiếm tiêu chuẩn dường như thất bại thảm hại:

Kiểm tra heuristic/đích sau khi sự việc xảy ra là không thỏa đáng

Tìm kiếm so với lập kế hoạch (tiếp)

Các hệ thống lập kế hoạch thực hiện những việc sau:

- 1) mở rộng biểu diễn hành động và đích để cho phép lựa chọn
- 2) chia để trị bằng cách đặt các mục tiêu con (subgoaling)
- 3) nói lỏng yêu cầu xây dựng các giải pháp tuần tự

	Tìm kiếm	Lập kế hoạch
Trạng thái	Cấu trúc dữ liệu Lisp	Các câu logic
Hành động	Mã Lisp	Tiền điều kiện/kết quả
Đích	Mã Lisp	Câu logic (phép hội)
Kế hoạch	Chuỗi từ S_0	Các ràng buộc trên các hành động

Lập kế hoạch trong phép tính tình huống

$PlanResult(p, s)$ là tình huống kết quả từ việc thực thi p trong s

$$PlanResult([], s) = s$$

$$PlanResult([a|p], s) = PlanResult(p, Result(a, s))$$

Trạng thái ban đầu $At(Home, S_0) \wedge \neg Have(Milk, S_0) \wedge \dots$

Hành động như các tiên đề Trạng thái kế tiếp

$$Have(Milk, Result(a, s)) \Leftrightarrow$$

$$[(a = Buy(Milk) \wedge At(Supermarket, s)) \vee (Have(Milk, s) \wedge a \neq \dots)]$$

Truy vấn

$$s = PlanResult(p, S_0) \wedge At(Home, s) \wedge Have(Milk, s) \wedge \dots$$

Giải pháp

$p = [Go(Supermarket), Buy(Milk), Buy(Bananas), Go(HWS), \dots]$

Khó khăn chính: phân nhánh không bị ràng buộc, khó áp dụng các heuristic

Các toán tử STRIPS

Mô tả các hành động được sắp xếp gọn gàng, ngôn ngữ bị hạn chế

HÀNH ĐỘNG (ACTION): $Buy(x)$

TIỀN ĐIỀU KIỆN (PRECONDITION): $At(p), Sells(p, x)$

HIỆU ỨNG (EFFECT): $Have(x)$

[Lưu ý: điều này trừu tượng hóa nhiều chi tiết quan trọng!]

Ngôn ngữ bị hạn chế $\Rightarrow$ thuật toán hiệu quả

Tiền điều kiện: phép hội của các literal khẳng định

Hiệu ứng: phép hội của các literal

Không gian trạng thái so với không gian kế hoạch

Tìm kiếm tiêu chuẩn: nút = trạng thái thế giới cụ thể

Tìm kiếm lập kế hoạch: nút = kế hoạch cục bộ (partial plan)

Định nghĩa: điều kiện mở (open condition) là một tiền điều kiện của một bước chưa được hoàn thành

Các toán tử trên các kế hoạch cục bộ:

thêm một liên kết từ một hành động hiện có đến một điều kiện mở

thêm một bước để hoàn thành một điều kiện mở

sắp xếp một bước so với bước khác

Chuyển dần từ các kế hoạch không đầy đủ/mơ hồ sang các kế hoạch đầy đủ, chính xác

Các kế hoạch được sắp xếp cục bộ

Một kế hoạch là đầy đủ (complete) khi và chỉ khi mọi tiên điều kiện đều đạt được

Một tiên điều kiện đạt được (achieved) khi và chỉ khi nó là hiệu ứng của một bước trước đó

và không có bước có khả năng can thiệp (possibly intervening) nào hủy bỏ nó

Phác thảo thuật toán POP

function POP(*initial*, *goal*, *operators*) **returns** *plan*

plan $\leftarrow$ MAKE-MINIMAL-PLAN(*initial*, *goal*)

loop do

if SOLUTION?(*plan*) **then return** *plan*

S_{need}, *c* $\leftarrow$ SELECT-SUBGOAL(*plan*)

CHOOSE-OPERATOR(*plan*, *operators*, *S_{need}*, *c*)

RESOLVE-THREATS(*plan*)

end

function SELECT-SUBGOAL(*plan*) **returns** *S_{need}*, *c*

chọn một bước kế hoạch *S_{need}* từ STEPS(*plan*)

với một tiền điều kiện *c* chưa đạt được

return *S_{need}*, *c*

Thuật toán POP (tiếp)

procedure CHOOSE-OPERATOR($plan, operators, S_{need}, c$)

choose chọn một bước S_{add} từ $operators$ hoặc STEPS($plan$) có hiệu ứng là c

if nếu không có bước như vậy **then fail**

thêm liên kết nhân quả $S_{add} \xrightarrow{c} S_{need}$ vào LINKS($plan$)

thêm ràng buộc thứ tự $S_{add} \prec S_{need}$ vào ORDERINGS($plan$)

if nếu S_{add} là một bước mới được thêm từ $operators$ **then**

thêm S_{add} vào STEPS($plan$)

thêm $Start \prec S_{add} \prec Finish$ vào ORDERINGS($plan$)

procedure RESOLVE-THREATS($plan$)

for each với mỗi S_{threat} đe dọa liên kết $S_i \xrightarrow{c} S_j$ trong LINKS($plan$) **do**

choose chọn hoặc

Hạ hạng (Demotion): Thêm $S_{threat} \prec S_i$ vào ORDERINGS($plan$)

Thăng hạng (Promotion): Thêm $S_j \prec S_{threat}$ vào ORDERINGS($plan$)

if not nếu không CONSISTENT($plan$) **then fail**

end

POP là đúng đắn, đầy đủ và có hệ thống (không lặp lại)

Mở rộng cho phép tuyến, lượng từ phổ dụng, phủ định, điều kiện

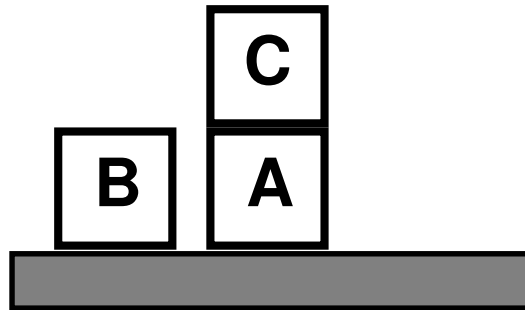
Đè bẹp (Clobbering) và thăng hạng/hạ hạng (promotion/demotion)

Một clobberer là một bước can thiệp tiềm năng phá hủy điều kiện đạt được bởi một liên kết nhân quả. Ví dụ, $Go(Home)$ đè bẹp $At(HWS)$:

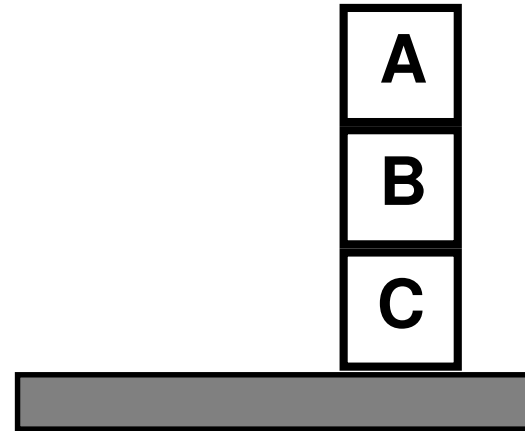
Hạ hạng (Demotion): đặt trước $Go(HWS)$

Thăng hạng (Promotion): đặt sau
 $Buy(Drill)$

Ví dụ: Thế giới khối (Blocks world)



Start State



Goal State



Ví dụ (tiếp)

